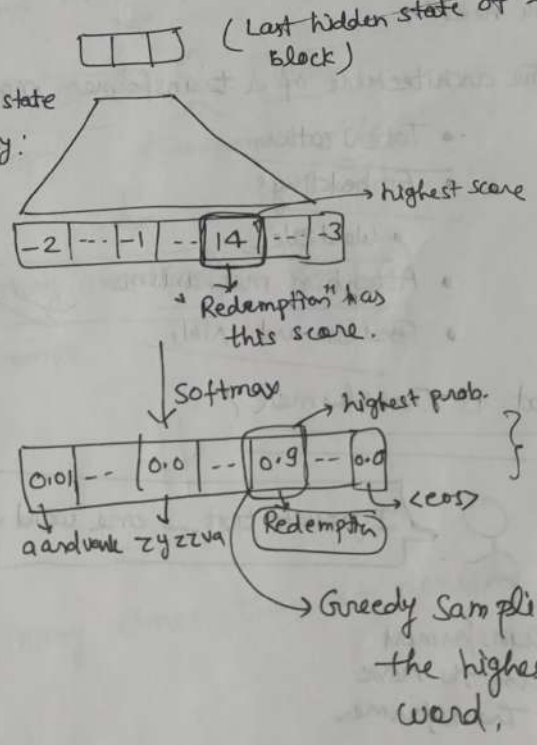
↳ To capture relationships in data is not able to do by just one block, so one block's O/P is fed into another block & so on.

↳ For language models., one hidden state that we're focused on the most is the last one.

⇒ Turning computation into language (O/P projection)



⇒ Transformers can do anything. They can chat with you, they can answer questions, they can follow commands, they can build stories/poems or they can write code.

[write code to build chatbots by searching youtube].

→ Transformers require huge datasets & a lot of computing power to be trained properly but they have simple architecture.

⇒ Transformers was introduced in paper "Attention is All you need".

- The architecture of a transformer model:
  - Tokenization
  - Embeddings
    - Word2Vec
  - Attention mechanisms.
  - Feedforward NNs.

- What is Transformer?

I generate text... one word at a time

Transformer from the movie Transformers

Ex.

① Hello, how are you

Prompt

② Write a story

Prompt

Write a story

Prom

Write a st

Pro

↳ If it does

a story.

Ex.

① Hello. how are you — Prompt → Transformers → doing — O/P.

② Write a story. — Prompt → (Transformers) → Once — O/P

⇓

Write a story. Once — Prompt → (Transformers) → Upon — O/P

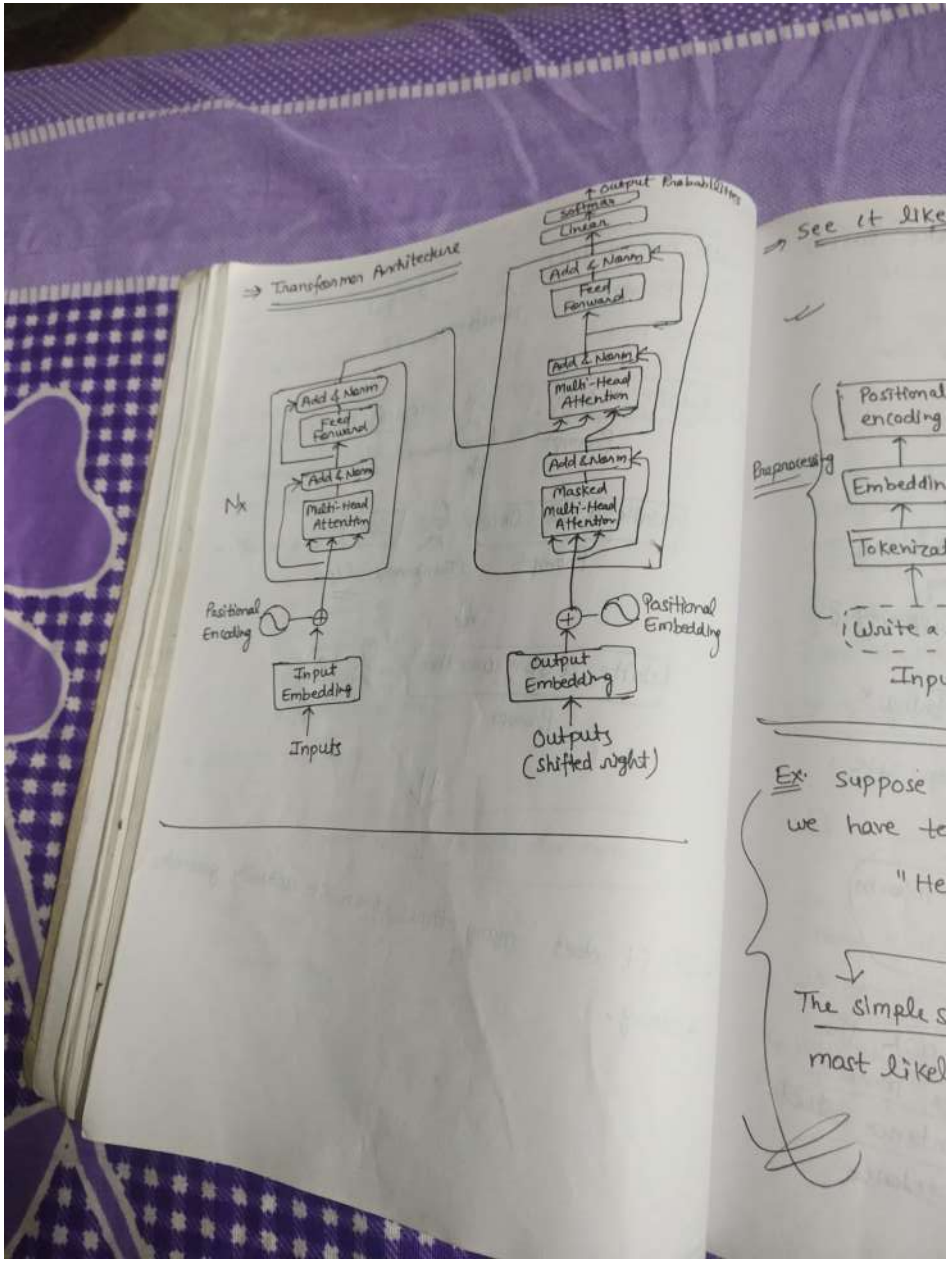
⇓

Write a story. Once Upon — Prompt → a — O/P.

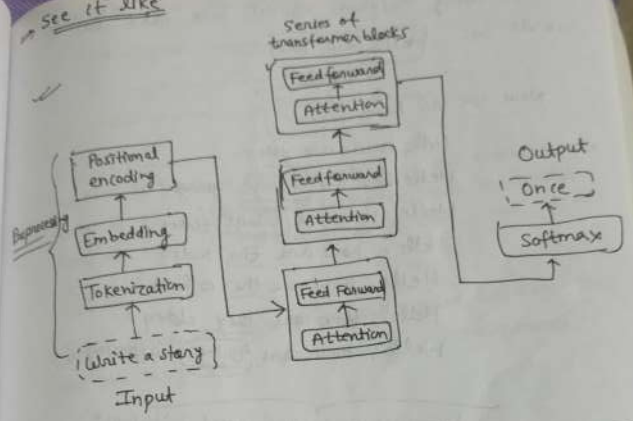
⇓

- - -

↳ If it does many times then it actually generates a story.


Transformer Architecture
Output Probabilities
Softmax
Linear
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Add & Norm
Masked Multi-Head Attention
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Nx
Positional Encoding
Positional Embedding
Input Embedding
Output Embedding
Inputs
Outputs (shifted right)

→ See it like



Ex. suppose we are texting a message to a friend & we have to predict the next word for :-

"Hello, how are ____".

word.

The simple sol^n. using ~~bigram~~ 1-gram model, we have to find most likely next word for "are", like,

--- are you ---
--- are sad --
-- are? ---
--- are happy
-- are ready ---
-- are happy
-- are free --

2 times., so suggest next word as "happy". but that doesn't always work very well

But it makes sense.
Now, using trigram model, we take last 2 words as Hello, how are
3 words.

Now, we can have.
Hello, how are you?
Hello, how are things going?
Hello, how are things today?
Hello, how are the kids?
Hello, how are the others?
Hello, how are they doing?
Hello, how are things happening?

here, most common word is "things"

↳ Now, to make it more powerful, we can use 10-grams.

Ex. Hello, today I would like to inform you that the.

10 words.

but the prob^n is, more words we pick, the more educated the guess is, but also the more words we pick, the less likely that the sentence could have appeared before. [bcoz may be sentence appeared 1st time].

⇒ We need

↳ We use

Hello, how are

↳ NN is trai
time & to bui
long text
Transformer

↳ ~~First we se~~
~~are the key~~

⇒ Tokenization

↳ It breaks

Ex. Write

There is also a

⇒ Embeddings.

↳ How do we build

↳ There are many

⇒ We need something more advanced.

↳ We use Neural Network (NN) as.

Hello, how are → [Neural Network] → your

↳ NN is trained to remember a few words at a time & to build short sentences. not to build long text & to build long text, we need a Transformer.

~~↳ First we see how NN works here because they are the key to understanding a transformer.~~

⇒ Tokenization

↳ It breaks everything into units.. A unit is a word.

Ex. [Write a story.] ⟹ [Write] [A] [story] [.]

[doesn't] ⟹ [does] [n't]

There is also a science about Tokenizat^n [see Jay's blog]

⇒ Embeddings.

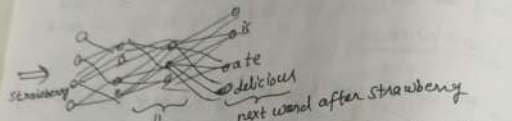
↳ How do we build embeddings?

↳ There are many ways. I am going to tell with "Neural Networks"

Eg. we have

Word	Numbers	
Strawberry	5	4
Apple	5	5
Castle	2	1

} very similar



It gives 3 numbers = 1 3 2 = embedding for strawberry
(Similar process for apple)

So, we have,

Word	Numbers		
Strawberry	1	3	2
Apple	1.1	2.9	2.2

} Very similar

↳ NN has layers & each layer understands deeper & deeper properties of the word

↳ When we train a NN to guess the next word, it has to understands words pretty well, it means these layers must capture properties of the word.

↳ This is a pretty common method to find embeddings & it is called Word to Vector (word2Vec).

⇒ Positional E

↳ It is very Imp

write a

because matter. S thing in different just learn

write a

Perturb words in these directions

write

a

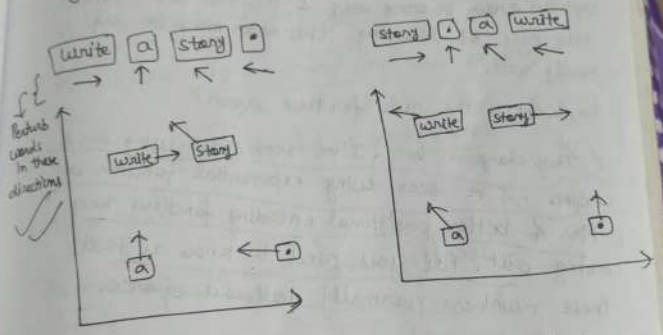
↳ Now when different

Positional Encoding.

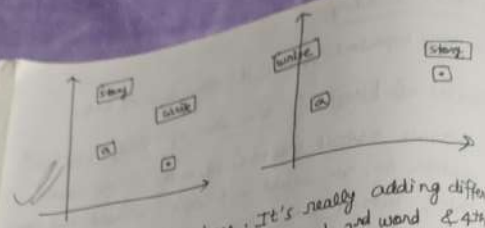
↳ It is very important because it takes care of order.

"write a story ." is not same as "story . a write"

because output matters, so, order of the words matter. So, we would not want to feed the same thing into a NN with 2 sentences in two different order. So, what to do?. Well, we have just learned embeddings.



↳ Now when the words move now we get different embeddings for these 2 diff sentences.



↳ There is no semantics here. It's really adding different numbers to the 1st word, 2nd word, 3rd word & 4th word etc., in such a way that now the embedding captures order in some way & then the NN would take care of learning this order more. It works really well.

↳ What is the rule for these arrows?

They change a lot; I've seen them using sins & cosines. I've seen using exponential function but better & better positional encoding functions keep coming out. All you need to know as that these numbers normally follow a sequence.

For ex: → ↑ ↖ ↙ ↙ ↓ ↘ ↘ → →

This sequence of arrows that rotate slower & slower every time are one that works as positional encoding but if you want to remember one thing about positional encoding, you just find some kind of sequence that you add to each one of the words in the sentence in order

to move them ar

⇒ Attention

↳ It is crucial in
made them work
really captures. t
times in the a
of the Transform
layers.

↳ Consider Attent
that appear

Ex: Please b

appl

↳ Using context

The bear ate

Question is:

Well, we don
next word.
"it" refers t
Than it refe

to move them around & perturb the order.

⇒ Attention

↳ It is crucial in transformer model. It is the step that made them work really well because it's the step that really captures the context. Attention can appear many times in the architecture because it appears in each one of the Transformer blocks followed by some Feed Forward layers.

↳ Consider Attention as gravitational pull b/w words that appear in the same sentence or in the same context.

Ex Please buy an apple and an orange

apple unveiled the new phone

influence by

↳ Using context

The bear ate the honey because it was ____

Question is: Does "it" refers to "honey" or "bear"?

Well, we don't know because it depends on the next word. If the blank word is hungry then "it" refers to "bear" & if the blank word is "delicious" then it refers to "honey"